

Advanced WordPress Form Builder Plugin

Technical Plan, Requirements, Architecture and Implementation Approach

Prepared for plugin planning and developer implementation

Generated 30 May 2026

Core Architecture Decision

The form itself should be built in a dedicated WordPress admin drag-and-drop builder. Elementor should not own the form structure. Elementor should select a saved form and expose styling controls based on the selected form feature map.

Layer	Responsibility	Should Not Do
WordPress Admin Form Builder	Build forms, steps, fields, repeaters, validation, conditions, actions, mappings, tests, uploads and submission settings.	Should not handle frontend page styling like Elementor spacing, typography, borders and shadows.
Plugin Database	Store form config, normalized config, feature map, versions, submissions, uploads, logs and test results.	Should not rely on Elementor page JSON as the only source of truth.
Elementor Widget	Select a saved form, render it and expose style controls based on available features.	Should not build form structure, logic, validation, routing or actions.
Frontend Renderer	Render the saved form, run conditions, uploads, validation, routing and AJAX/REST submissions.	Should not trust browser-only validation or client-supplied structure.

Contents

1. 1. Project Objective and Principles
2. 2. Plugin Structure
3. 3. Database Requirements
4. 4. Admin Drag-and-Drop Builder
5. 5. Admin Asset Loading Rules
6. 6. Form Settings
7. 7. Step Builder Requirements
8. 8. Field Builder Requirements
9. 9. Repeater and Grid Requirements
10. 10. Conditional Logic and Routing
11. 11. Validation, Sanitization and Escaping
12. 12. Instant Uploads and Multi-File Uploads
13. 13. Actions on Submit
14. 14. ACF, Meta, Taxonomy and Product Mapping
15. 15. Elementor Widget and Dynamic Styling Sections
16. 16. Frontend Engine and JavaScript Events
17. 17. Security, Nonce Refresh and Anti-Spam
18. 18. Submissions, Logs and Admin Views
19. 19. Form Test Runner
20. 20. Sidebar Filter Builder
21. 21. Developer Hooks and Extensibility
22. 22. Implementation Phases
23. 23. Acceptance Checklist

1. Project Objective and Principles

The goal is to build a modular WordPress plugin that can create advanced forms, multi-step onboarding flows, login/register flows, upload forms, mapped content update forms and sidebar filter interfaces. The form builder should be powerful enough to replace several form and query plugins while remaining developer-friendly and performant.

Principle	Requirement	Reason
Admin-first form building	All form structure and logic is built in a WordPress admin drag-and-drop UI.	A complex form builder is too large for the Elementor sidebar and needs a full admin workspace.
Database source of truth	Save form configuration, normalized configuration and feature map in plugin tables.	Allows reuse, versioning, testing, secure validation and faster rendering.
Elementor for display and style	Elementor selects the saved form and controls the visual styling sections.	Elementor remains focused on layout and styling rather than form logic.
Strict separation of assets	Admin assets only load in admin plugin pages; frontend assets only load when the form exists.	Prevents performance issues and avoids unnecessary global scripts.
Server-side enforcement	All frontend validations, conditions and actions must be verified server-side.	Prevents DOM tampering, bypassed validation and unsafe submissions.
Template override support	Every form element should render through a template that a child theme can override.	Developers can customize markup without editing the plugin core.
Modular actions	Post-submit actions must be registered and run through an action runner.	Keeps login, register, ACF, meta, product and webhook flows clean and extensible.

2. Plugin Structure

The plugin should be split by responsibility. Avoid placing builder logic, frontend rendering, submission processing and integrations in the same class.

```
advanced-form-builder/  
  advanced-form-builder.php  
  includes/  
    Core/           Plugin bootstrap, loader, hooks, capabilities, assets  
    Admin/          Forms UI pages, submissions page, test page, settings page  
    Builder/        Drag/drop builder, config normalizer, feature map generator  
    Elementor/      Widget to select saved form and expose style controls  
    Forms/          Renderer, validator, sanitizer, condition engine, action runner  
    Fields/         Field classes: text, email, select, file, repeater, range, HTML  
    Actions/        Login, register, update ACF, update meta, redirect, webhook  
    Uploads/        Instant upload, temp upload, finalization, cleanup  
    Filters/        Sidebar filter builder and query runner  
    Database/       Tables and repositories  
    REST/          Submit, upload, nonce refresh, live checks, filter queries  
    Utilities/      Array, security, template, validation, date, user, post helpers  
  assets/  
    admin/          Admin-only builder CSS/JS  
    frontend/       Frontend form CSS/JS  
    elementor/      Elementor editor CSS/JS  
    vendor/         Select2, validation library, drag/drop library if used  
  templates/  
    form.php  
    step.php
```

fields/
filters/

Module	What it should do	Important Classes
Core	Load the plugin, register hooks, setup constants, register activation/deactivation routines and central asset registration.	Plugin, Loader, Hooks, Assets, Installer
Admin	Provide WordPress admin pages for forms, form builder, submissions, tests and settings.	Forms_Page, Edit_Form_Page, Submissions_Page, Tests_Page, Settings_Page
Builder	Manage drag/drop form state, normalize saved configs, validate configs and generate feature maps for Elementor styling.	Builder_Config_Normalizer, Feature_Map_Generator
Elementor	Register a form widget that selects a saved form and exposes style controls based on feature map.	Form_Widget, Widget_Style_Controls
Forms	Render forms, evaluate conditions, run validation, sanitize submissions and execute submit actions.	Form_Renderer, Form_Validator, Form_Action_Runner
Fields	Provide field definitions, default settings, render templates and field-specific validation rules.	Abstract_Field and individual field classes
Actions	Provide modular submit actions such as update ACF, update meta, register user, login, redirect and webhook.	Abstract_Action and action classes
Uploads	Handle instant uploads, temporary file state, validation, final attachment commit and cleanup.	Upload_Controller, Upload_Validator, Upload_Finalizer
REST	Expose endpoints for submit, upload, nonce refresh, live checks, tests and filtering.	Routes, Submit_Form_Route, Upload_File_Route
Utilities	Provide reusable helper methods without mixing business logic.	Security_Utils, Template_Utils, Validation_Utils

3. Database Requirements

The plugin should store form structure separately from Elementor. Elementor should only store the selected form ID and page-specific style overrides.

Table	Purpose	Key Fields
wp_afb_forms	Stores the current saved form configuration.	id, form_uuid, form_name, form_type, status, config_json, normalized_config_json, feature_map_json, version, created_by, updated_by, created_at, updated_at
wp_afb_form_versions	Stores a version history for rollback and debugging.	id, form_id, version, config_json, normalized_config_json, created_by, created_at
wp_afb_submissions	Stores form submissions and action results.	id, form_id, form_uuid, form_instance_id, user_id, status, source_url, submitted_data_json, sanitized_data_json, validation_result_json, action_results_json, created_at
wp_afb_uploads	Tracks instant uploads before and after final submission.	id, form_id, field_id, user_id, temp_token, attachment_id, original_filename, mime_type, file_size, upload_status, is_committed, expires_at
wp_afb_test_logs	Stores form test runs, expected results, actual results and cleanup records.	id, form_id, test_group_id, test_name, expected_result_json, actual_result_json, status, created_records_json, cleanup_status, created_at
wp_afb_audit_logs	Stores important admin and runtime events.	id, form_id, event_type, event_context_json, user_id, created_at

The normalized configuration should be generated from the builder config. The feature map should be generated from the normalized config and used by Elementor to decide which styling sections should appear.

4. Admin Drag-and-Drop Builder

The WordPress admin form builder should be a full-screen drag-and-drop interface. This is where forms are created and edited.

Area	What it should do	Requirements
Left Panel	Display draggable field and structure blocks.	Basic fields, choice fields, date/range fields, upload fields, repeater, grid, HTML, notice, hidden field, submit buttons.
Middle Canvas	Display the form as steps, sections, fields and repeaters.	Allow drag/drop reorder, move fields between steps, move fields into grids and move fields inside repeaters.
Right Panel	Show settings for the selected item.	General settings, label, placeholder, validation, conditions, mapping, upload rules, style-related options and advanced attributes.
Top Bar	Save, preview, run tests, view submissions, duplicate form and publish/unpublish.	Must indicate unsaved changes and validation issues in the builder configuration.
Config Validator	Check the form configuration before saving.	Detect missing field IDs, duplicate IDs, broken conditions, invalid routes, invalid mappings and missing action settings.
Preview Mode	Render a safe admin preview of the form.	Should use the real renderer but not submit real actions unless test mode is enabled.

5. Admin Asset Loading Rules

Admin assets must only load on admin screens used by this plugin. Frontend assets must only load when the form or filter widget is present.

Asset Context	Load When	Should Load
Admin builder	Only on admin.php?page=afb-edit-form or equivalent builder page.	Builder CSS, drag/drop JS, builder state JS, condition builder JS, validation builder JS, mapping builder JS.
Admin forms list	Only on the all forms list page.	Forms list CSS and small admin UI JS if needed.
Admin submissions	Only on submissions pages.	Submission table CSS and export/detail JS if needed.
Admin tests	Only on test runner pages.	Test runner JS, test logs CSS and cleanup controls.
Elementor editor	Only inside Elementor editor when the widget is registered.	Widget editor JS and CSS for dynamic style panel behavior.
Frontend form	Only on pages where the selected form is rendered.	Form engine, condition engine, validation engine, step router, upload manager only if needed.
Frontend filters	Only on pages where the sidebar filter widget is rendered.	Sidebar filter JS, query AJAX handler and filter CSS.

Do not enqueue Select2, upload scripts, range slider scripts or validation libraries globally. They should be registered globally but only enqueued if the form feature map requires them.

6. Form Settings

Each form should have global settings controlled from the WordPress admin builder. These settings define how the form behaves before field-level configuration is applied.

Setting Group	What it should do	Controls
Form identity	Set form name, internal key, UUID, status and description.	Form name, internal key, type, status, description.
Submission storage	Choose whether submissions should be saved to the database.	Store submissions on/off, retention period, export permission.
Submission endpoint	Choose REST endpoint submission as default.	REST submission enabled, fallback admin-ajax optional.
Logged-in requirement	Require user to be logged in before accessing or submitting the form.	Require login, allowed roles, allowed capabilities, redirect/login message.
Redirect behavior	Set default redirect URL or allow action-specific redirect.	Use global redirect, custom redirect URL, conditional redirect rules.
Nonce refresh	Refresh the form nonce before final submit to support cached sites.	Enable dynamic nonce refresh, retry once on nonce failure.
Anti-spam	Enable zero-UI spam protections.	Honeypot, time-to-submit, rate limit, repeated submission block.

Uploads	Control whether instant uploads are allowed and how long temp files last.	Enable uploads, max size, allowed MIME types, temp expiry.
Draft persistence	Allow saved progress for long forms.	LocalStorage, logged-in autosave, expiry time.
Developer events	Enable JS lifecycle events and optional console helper notices.	Event dispatch enabled, debug console notices enabled.

7. Step Builder Requirements

Steps define the flow of a form. Even a single-page form can internally use one step for consistency.

Step Feature	What it should do	Builder Controls
Step identity	Each step must have a unique step ID and display name.	Step ID, step name, internal description.
Step heading/description	Render optional heading and description above fields.	Heading, description, heading visibility.
Step buttons	Control previous, next and submit button text per step.	Previous text, next text, submit text, button visibility.
Step validation scope	Validate only visible required fields in the current step before next step.	Validate current step, validate hidden conditional fields on/off.
Step routing	Route to different next steps based on field values or API checks.	Conditional next step rules, default next step, fallback error.
Step conditions	Show or skip the step based on previous answers.	Show if, hide if, skip if, required to reach.
Step notices	Display notices before or after step fields.	Before notice, after notice, notice type.
Step layout	Support columns, grids and repeaters inside the step.	Layout mode, grid settings, custom class.

8. Field Builder Requirements

Every field should share a common base setting model and then expose type-specific settings. Field IDs must be stable because conditions, validation, actions and mapping rely on them.

Field Type	What it should do	Builder Controls
Text	Capture normal short text values.	Label, placeholder, default value, min/max length, regex, mapping target.
Email	Capture and validate email addresses.	Email validation, existing user check, unique email option, mapping target.
Password	Capture secure password values.	Password strength, min length, confirm password pairing, no database storage option.
Confirm password	Validate that a second password field matches the main password field.	Select main password field, custom mismatch message.
Textarea	Capture longer text content.	Rows, max length, allowed HTML on/off, post content mapping.
Number	Capture numeric values.	Min, max, integer/decimal, step, mapping target.
URL	Capture URL values.	URL validation, allowed domain options, mapping target.
Phone	Capture phone values.	Pattern, country mode, min/max length, mapping target.
Select	Render standard select dropdown.	Options, default, required, dynamic source, mapping target.
Select2	Render enhanced searchable select.	Single/multi, AJAX source, taxonomy/user/post/product source, max selections.
Checkbox group	Capture one or more selected values.	Options, min selected, max selected, conditional targets, mapping target.
Radio group	Capture one selected value.	Options, default, required, conditional targets, mapping target.
Date	Capture date values.	Date format, after today, before today, over 18, min/max date, mapping target.
Range slider	Capture numeric range or single numeric value.	Min, max, step, prefix, suffix, live value display, mapping target.
File upload	Upload one file instantly before submit.	Allowed types, max size, required, map to attachment/ACF/product

		image.
Multi-file upload	Upload multiple files instantly before submit.	Min/max files, reorder, gallery mapping, allowed types, max size.
Hidden field	Store dynamic tokens or secure hidden values.	Token type, signed value, query param, user meta, post meta, timestamp.
HTML block	Display static or dynamic HTML inside the form.	HTML content, allowed tags, conditional display, dynamic placeholders.
Notice block	Display info, success, warning or error notice.	Notice title, text, type, position, conditional display.
Repeater	Allow multiple rows of nested fields.	Min rows, max rows, row fields, add/remove labels, validation per row.
Grid/column break	Structure fields into columns without saving data.	Columns, responsive stacking, start/end wrapper behavior.

9. Repeater and Grid Requirements

Repeaters and grids are structural features that make the builder more powerful than a simple contact form system.

Feature	What it should do	Validation / Storage Requirements
Repeater field	Allow the admin to place fields inside a repeatable row structure.	Save submitted values as structured arrays and validate each row.
Nested field naming	Use deterministic names so PHP can parse nested values safely.	Example: <code>afb[form_id][step_id][repeater_id][row_index][field_id]</code> .
Minimum rows	Prevent submission unless a minimum number of rows exists.	Client-side and server-side validation required.
Maximum rows	Prevent too many rows being added or submitted.	Reject extra rows server-side even if DOM is manipulated.
Repeater conditions	Show fields inside each row based on row values.	Conditions must scope to the current row unless configured globally.
Grid/column break	Wrap fields into 2, 3 or custom column layouts.	Does not submit data; affects only rendering and Elementor style targets.
Repeater mapping	Map repeater data to ACF repeater fields, post meta arrays, user meta arrays or product gallery/attributes where supported.	Must validate destination type before update.

10. Conditional Logic and Routing

Conditions should be a shared engine used by the admin builder preview, frontend JavaScript and server-side final submission validation.

Condition Type	What it should do	Examples
Show/hide field	Show or hide a field based on another field value.	Show business name if <code>account_type = business</code> .
Show/hide HTML	Show an HTML section based on a value.	If <code>first_name = Linden</code> , show Welcome Linden.
Replace dynamic span	Replace placeholders in HTML with live field values.	Welcome, <code>{first_name}</code> .
Step routing	Send the user to a different next step based on answers.	Creator goes to Creator Details; customer goes to Customer Details.
Button blocking	Disable next/submit until a condition passes.	Cannot proceed while username check is failed.
Action condition	Run a submit action only when a condition is true.	Only update product fields if <code>product_mode = yes</code> .
Validation condition	Make a field required only under certain conditions.	Require company number if business account is selected.

Supported operators should include equals, not equals, contains, does not contain, starts with, ends with, greater than, less than, is empty, is not empty, checked, not checked, date after, date before, age over, age under, file uploaded, API check passed and API check failed.

11. Validation, Sanitization and Escaping

The plugin must not rely on browser or JavaScript validation only. Every validation rule configured in the builder must be mirrored and enforced in PHP during final submission.

Validation Rule	What it should do	Common Use
Required	Reject empty required fields.	All field types.
Email	Validate email format and optionally uniqueness.	Email field, registration flow.
URL	Validate URL and optionally allowed domains.	URL field.
Number min/max	Ensure numeric values are inside allowed range.	Number and range fields.
Date after today	Reject dates not after today.	Date field.
Age over 18	Calculate age from date of birth and require minimum age.	Date of birth field.
Password strength	Validate password length and complexity rules.	Password field.
Confirm password	Validate two password fields match.	Register/reset forms.
File type	Allow only configured MIME types and extensions.	Upload fields.
File size	Reject files over max size.	Upload fields.
Checkbox min/max	Validate number of selected checkboxes.	Checkbox groups.
Value must not equal X	Reject specific blocked values.	Username, reserved terms, route blockers.
API check required	Require live check to pass before next step or submit.	Username availability, SKU availability.

Data Type	Sanitization	Escaping on Output
Text	<code>sanitize_text_field</code>	<code>esc_html</code> or <code>esc_attr</code>
Textarea	<code>sanitize_textarea_field</code> or <code>wp_kses_post</code> when allowed	<code>esc_textarea</code> or <code>wp_kses_post</code>
Email	<code>sanitize_email</code>	<code>esc_html</code> / <code>esc_attr</code>
URL	<code>esc_url_raw</code>	<code>esc_url</code>
Number	<code>absint</code> , <code>intval</code> or <code>floatval</code> depending on config	<code>esc_html</code>
Checkbox/select/radio	Sanitize each selected value against allowed options.	<code>esc_html</code> / <code>esc_attr</code>
Hidden	<code>sanitize_text_field</code> plus signed token verification where sensitive.	<code>esc_attr</code>
HTML block	<code>wp_kses_post</code> or strict allowed tags.	<code>wp_kses_post</code>
File upload	Validate temp token, MIME, extension, ownership and final attachment ID.	<code>esc_url</code> / <code>esc_html</code> for labels

12. Instant Uploads and Multi-File Uploads

Uploads should happen immediately when a file is selected. The final submit should only commit already uploaded temporary files to the submission/action context.

Upload Feature	What it should do	Implementation Requirement
Instant upload	Upload selected file immediately through REST before final submit.	Return temp token, attachment ID or secure file hash.
Temporary status	Mark upload as temporary until form submission succeeds.	Store in <code>wpafb_uploads</code> with <code>is_committed</code> = 0.
Finalization	On successful submit, bind upload to submission and action target.	Set <code>is_committed</code> = 1 and map to ACF/media/product if configured.
Cleanup	Delete abandoned temporary uploads after expiry.	Scheduled cleanup task.
Multi-file	Support multiple files with progress and ordering.	Store array of upload tokens and order index.
Upload validation	Validate in JS for UX and in PHP for security.	MIME, extension, size, permission, nonce, form ID and field ID.
Upload mapping	Allow upload fields to update ACF image/file/gallery, post featured image, user meta, product gallery or downloadable files.	Destination must match field type and selected mapping.

13. Actions on Submit

The form builder must include an Actions on Submit panel. Admin users should add one or more actions, configure each action, optionally conditionally run it, and order the actions. Each action should return a success/failure result that is stored against the submission.

Action	What it should do	Admin Controls	Implementation Notes
--------	-------------------	----------------	----------------------

Save Submission	Always or optionally save the submitted and sanitized data into wp_afb_submissions.	Storage on/off, excluded fields, sensitive fields to mask.	Submission ID stored; action result attached to response.
Update ACF Field	Update a selected ACF field using submitted values.	Select target object type, target ID source, ACF field group, ACF field, source form field, append/replace mode.	Use ACF update_field when available; validate field type before update.
Update ACF Repeater	Map form repeater rows to an ACF repeater field.	Select ACF repeater field, map child form fields to subfields, replace/append mode.	Build array matching ACF subfield keys and update through ACF API.
Update User Meta	Update metadata for the current user, created user or selected user.	Select user target, meta key, source field, replace/append mode.	Use update_user_meta after permission checks.
Update Post Meta	Update metadata for a selected post, current post or created post.	Select post target, meta key, source field, replace/append mode.	Use update_post_meta after validating user can edit target when required.
Update Post Title	Update a post title from a field value.	Select post target and source field.	Use wp_update_post with sanitized title.
Update Post Content	Update post content/description from a textarea or HTML-approved field.	Select post target and source field.	Use wp_update_post; validate allowed HTML rules.
Create Post	Create a new post from submitted values.	Post type, status, title field, content field, excerpt field, author source.	Return created post ID and expose it to later actions.
Update Taxonomy Terms	Set, append or remove terms on a target post/product.	Select taxonomy, source field, target post, append/replace.	Use wp_set_object_terms with term validation.
Register User	Create a WordPress user from the form.	Email field, username field, password field, role, meta mapping, auto-login option.	Use wp_insert_user or wp_create_user; return created user ID.
Login User	Authenticate and log in a user.	Username/email field, password field, remember me, redirect option.	Use wp_signon and return auth errors safely.
Lost Password	Trigger a lost password/reset flow.	Email/username field, success message, redirect option.	Use WordPress reset flow hooks where possible.
Set User Role	Set or add a role after successful completion.	Target user, role, replace/add mode, conditional rules.	Run only after user exists and permission is valid.
Update WooCommerce Product	Update product fields from the form.	Select product target, product field, source form field, product type options.	Use WooCommerce CRUD methods; validate WooCommerce is active.
Create WooCommerce Product	Create a product from form fields.	Product type, title, description, price, stock, category, image/gallery fields.	Use WC_Product classes and return product ID.
Send Email	Send an admin/user email after submit.	To, CC, BCC, subject, message template, attachments.	Use wp_mail; support tokens from submitted data and created records.
Webhook	Send sanitized payload to an external URL.	URL, method, headers, body mapping, secret/signature.	Use wp_remote_post/request; log response status.
Redirect	Redirect after success.	Static URL, dynamic URL, conditional redirect, query parameters.	Return redirect_url in AJAX response.
Custom PHP Hook	Run developer-registered custom logic.	Hook/action key, payload selection, condition rules.	Use do_action/apply_filters and log result.

14. ACF, Meta, Taxonomy and Product Mapping

Mapping should be available either as field-level mapping or as submit actions. For complex updates, prefer action-level mapping because the admin can order and conditionally run actions.

14.1 ACF Mapping Requirements

ACF Item	What it should do	Admin/Server Requirement
ACF availability check	Detect if ACF is active before showing ACF actions.	If ACF is inactive, show disabled state and explanation.
Target object type	Allow the admin to choose where the ACF field is updated.	Current post, selected post, created post, current user, created user, options page if allowed.
Field group selector	Load field groups based on selected target object type.	Use AJAX/REST in admin builder to fetch available groups.
Field selector	After selecting a field group, allow choosing a specific field.	Show field label, name, key and type.
Source selector	Select which form field provides the value.	Only show compatible form fields where possible.
Compatibility validation	Validate that destination ACF type can accept the source value.	Image/file/gallery fields require upload field; repeater requires repeater mapping.
Update mode	Choose replace, append or merge where supported.	Text fields replace; galleries may append; repeaters may replace/append.
Runtime update	Update the selected ACF field after submit.	Use update_field with field key when possible.

14.2 WordPress Meta and Post Mapping Requirements

Mapping Type	What it should do	Controls
User meta	Map a submitted field to user meta.	Select current user, created user, target user field or fixed user ID; choose meta key.
Post meta	Map a submitted field to post meta.	Select current post, created post, target post field or fixed post ID; choose meta key.
Post title	Update or create post title from a source field.	Select title source and target post.
Post content	Update post content from a textarea/HTML-approved field.	Select source and allowed HTML behavior.
Post excerpt	Update post excerpt from a field.	Select source field.
Featured image	Set featured image from upload field.	Select upload field and target post.
Taxonomy terms	Map checkbox/select values to taxonomy terms.	Select taxonomy, source field, append/replace mode.

14.3 WooCommerce Product Mapping Requirements

Product Mapping	What it should do	Controls
Product target	Allow update of current product, selected product or created product.	Target selector.
Product title	Map text field to product title.	Source field selector.
Product description	Map textarea/HTML-approved field to long description.	Source field selector.
Short description	Map textarea field to short description.	Source field selector.
Regular price	Map number/range field to regular price.	Source field selector with numeric validation.
Sale price	Map number/range field to sale price.	Source field selector with numeric validation.
Stock status	Map select/radio to stock status.	Allowed values: instock, outofstock, onbackorder.
SKU	Map text field to SKU and optionally run live uniqueness check.	Source field plus live API check option.
Product image	Map file upload field to featured image.	Upload field selector.
Product gallery	Map multi-file upload field to gallery IDs.	Multi-file upload selector.
Product categories/tags	Map checkbox/select to WooCommerce taxonomies.	Taxonomy selector and append/replace mode.
Product attributes	Map fields to product attributes.	Attribute selector, value source and visible/variation flags.

15. Elementor Widget and Dynamic Styling Sections

The Elementor widget should select a saved form and use Elementor controls for all styling. Form logic and structure should remain inside the WordPress admin builder.

Elementor Feature	What it should do	Controls / Behavior
Select saved form	Allow the page builder to choose one saved form.	Dropdown listing published forms from wp_afb_forms.
Display mode	Choose normal page render, compact mode or possibly popup-compatible render.	Display options only; no structure editing.
Feature map detection	Read selected form feature map to know which style sections are useful.	has_steps, has_repeaters, has_uploads, has_select2, has_range, has_html, has_notices.
Style sections appear based on options	Only show style panels relevant to selected form features.	If no upload fields, hide upload style section. If no repeater, hide repeater style section.
Page-specific overrides	Store Elementor style overrides per widget instance.	Does not modify the saved form config.
Form wrapper styles	Style the outer form container.	Background, padding, margin, border, radius, shadow.
Input styles	Style input and textarea elements.	Typography, border, radius, padding, placeholder color, focus state.
Label styles	Style labels if the form has visible labels.	Typography, color, spacing, required marker.
Button styles	Style previous, next, submit and repeater buttons.	Typography, colors, padding, hover, disabled state.
Validation message styles	Style messages below fields.	Error color, success color, spacing, icon, typography.
Step styles	Style step containers and step navigation if form has steps.	Step background, spacing, progress indicator, active state.
Repeater styles	Style repeater rows if form has repeaters.	Row border, spacing, add/remove button styles.

Upload styles	Style upload fields if uploads exist.	Dropzone, progress bar, file list, remove button.
Range styles	Style range slider if range fields exist.	Track, handle, value display.

16. Frontend Engine and JavaScript Events

The frontend engine should initialize per form instance. It should not scan the whole page unnecessarily and should not load modules that the selected form does not need.

Frontend Module	What it should do	Load When
FormEngine	Initialize the form instance, state and submission flow.	Any form render.
ConditionEngine	Show/hide fields, HTML sections and buttons based on conditions.	When feature map has conditions.
ValidationEngine	Run client-side validation for UX before server validation.	When form has validation rules.
StepRouter	Handle next/previous navigation and conditional routing.	When form has steps.
UploadManager	Handle instant uploads, progress, remove and token storage.	When form has upload fields.
LiveCheckManager	Run debounce API checks for username, SKU or custom checks.	When form has live checks.
EventDispatcher	Dispatch lifecycle events and optional console helper messages.	When events are enabled.

JS Event	What it should do
afb:form:init	Fires when a form instance starts initializing.
afb:form:ready	Fires when the form is ready for interaction.
afb:step:changed	Fires after a successful step change.
afb:field:changed	Fires when a field value changes.
afb:validation:failed	Fires with validation errors.
afb:upload:started	Fires when an instant upload starts.
afb:upload:success	Fires when an upload succeeds.
afb:live-check:success	Fires when an API check passes.
afb:submit:started	Fires before final submit.
afb:submit:success	Fires after successful server response.
afb:submit:failed	Fires after failed server response.

17. Security, Nonce Refresh and Anti-Spam

Security must be part of the form engine rather than an afterthought. Every submission must be validated against the saved database config.

Security Feature	What it should do	Implementation Requirement
Dynamic nonce refresh	Fetch a fresh nonce right before final submit to avoid cached nonce failure.	POST /afb/v1/refresh-nonce returns nonce for form instance.
Server-side config lookup	Load saved normalized form config from database during submit.	Do not trust posted structure or hidden frontend config.
Unknown field rejection	Reject fields that are not in the saved form config.	Prevents injected values from being processed.
Conditional honeypot	Add a no-UX anti-spam field and reject if filled.	Must be obscure and per-form/per-instance where possible.
Time-to-submit	Reject submissions sent too quickly.	Compare generated timestamp and submit time.
Rate limit	Limit repeated submissions by user/IP hash/form ID.	Use transients or custom table if needed.
Permission checks	Validate user role/capability for protected forms and actions.	Required before user meta/post meta/product update.
Upload security	Validate nonce, token, ownership, MIME, extension, size and field rules.	Never trust file data from client only.
Redirect safety	Validate redirect URLs.	Only allow same-site by default unless external allowed.

18. Submissions, Logs and Admin Views

The admin should include enough visibility to debug forms and support clients after launch.

Admin View	What it should do	Required Details
All Forms	List saved forms with status and usage.	Form name, type, status, shortcode, Elementor usage, last updated.
Submissions List	List submissions with filters.	Form, user, status, date, source URL, action status.
Submission Detail	Show raw, sanitized and mapped data.	Submitted data, sanitized data, validation result, uploads, action results.
Upload Logs	Show upload status and abandoned temp files.	File name, field, user, status, expiry, attachment ID.
Audit Logs	Track major runtime/admin events.	Event type, form, user, context, date.
Settings	Configure global defaults.	Redirects, upload limits, anti-spam defaults, retention, debug events.

19. Form Test Runner

The test runner should allow developers/admins to simulate submissions and verify that fields, validation, actions and mappings behave as expected.

Test Type	What it should do	Expected Result Examples
Validation failure test	Submit invalid values and confirm the correct errors appear.	Email invalid; age under 18 blocked; required field missing.
Validation success test	Submit valid values and confirm no validation errors occur.	Expected status = pass.
Step routing test	Check that a specific input routes to the expected step.	account_type creator routes to creator_details.
Live API check test	Mock or run username/SKU check.	Taken value blocks next step; available value allows it.
ACF update test	Create a temporary post/user and confirm ACF field updates.	Expected ACF field value equals submitted value.
User meta test	Create a temporary user and update selected user meta.	Expected meta key exists with expected value.
Product mapping test	Create temporary product and map values.	Expected price/title/category/gallery updated.
Upload test	Upload a test file and verify finalization.	Expected attachment committed and mapped.
Role assignment test	Create/register user and apply role.	Expected user role equals configured role.
Cleanup test	Delete created test users, posts, products, submissions and attachments.	Expected cleanup_status = complete.

All tests should log expected result, actual result, pass/fail status and IDs of created records. Cleanup must use the logged IDs so test data can be safely removed.

20. Sidebar Filter Builder

The same builder concepts can power a sidebar filter system, but it should be a separate form type or separate module because it runs queries instead of submissions.

Filter Feature	What it should do	Controls
Filter form type	Create a saved filter configuration in the admin builder.	Post type, query target, result template, pagination settings.
Filter sections	Allow sections such as Categories, Price, Date, Tags and Search.	Collapsible section, default open/closed, field list.
Taxonomy filters	Filter posts/products by taxonomy terms.	Taxonomy selector, checkbox/radio/select mode.
Meta filters	Filter by post meta or ACF fields.	Meta key/ACF field selector, compare operator, field type.
Range sliders	Filter numeric values such as price or distance.	Min, max, step, query meta key.
Elementor Loop support	Update an Elementor loop/result area using AJAX.	Target selector, query ID, empty template, loading state.
Pagination	Return paginated filtered results.	Page size, AJAX pagination, load more option.
URL sync	Optionally sync active filters to the URL.	Query string mode and browser history update.
Active chips	Show selected filters above results.	Chip style and remove behavior.

21. Developer Hooks and Extensibility

The plugin should expose clean PHP hooks and JavaScript events so developers can extend it without editing core files.

Hook Type	Hook Name	Purpose
Filter	afb_form_config	Modify normalized form config before render.
Filter	afb_validation_rules	Modify validation rules.
Filter	afb_sanitized_submission_data	Modify sanitized data before actions.
Filter	afb_submission_actions	Register or modify available actions.
Filter	afb_redirect_url	Modify redirect URL.
Filter	afb_filter_query_args	Modify sidebar filter query args.
Action	afb_before_form_render	Run before form HTML is rendered.
Action	afb_after_form_render	Run after form HTML is rendered.
Action	afb_before_validation	Run before server validation.
Action	afb_after_validation	Run after server validation.
Action	afb_after_submission_save	Run after submission is saved.
Action	afb_after_action_run	Run after a submit action runs.
Action	afb_upload_completed	Run after upload finalization.

22. Implementation Phases

Phase	Build	Completion Criteria
Phase 1 - Foundation	Plugin bootstrap, loader, DB installer, admin menus, asset manager, REST route base, settings page.	Plugin activates, creates tables and shows admin pages.
Phase 2 - Admin Builder MVP	Drag/drop builder, form save/load, steps, basic fields, config normalizer and feature map.	Admin can create and save a basic multi-step form.
Phase 3 - Frontend Render MVP	Render saved form, basic validation, REST submit, save submission.	Elementor widget can select and render a saved form.
Phase 4 - Conditions and Routing	Conditional display, HTML visibility, step routing, blockers.	Multi-step conditional forms work.
Phase 5 - Full Validation/Sanitization	Client and server validation library, custom messages, sanitization/escaping map.	Final submit enforces all rules server-side.
Phase 6 - Instant Uploads	Upload endpoint, temp tokens, multi-file upload, finalization, cleanup.	Files upload before submit and map after submit.
Phase 7 - Submit Actions	Save submission, update ACF, update meta, user auth flows, redirect, email, webhook.	Actions run in order and log results.
Phase 8 - Elementor Styling	Dynamic style panels based on feature map and page-specific overrides.	Elementor fully controls form styling.
Phase 9 - Tests and Logs	Test runner, expected/actual logs, cleanup, admin visibility.	Admin can test forms and remove test data.
Phase 10 - Sidebar Filters	Filter builder, query engine, Elementor loop support, pagination, URL sync.	Saved filters can render and update result loops.
Phase 11 - Hardening	Security review, permissions, nonce refresh, anti-spam, performance audit.	Production-ready release candidate.

23. Acceptance Checklist

Area	Acceptance Requirement
Architecture	Forms are created in WordPress admin, not inside Elementor.
Architecture	Elementor widget selects a saved form and handles styling only.

Database	Forms, versions, submissions, uploads, test logs and audit logs have dedicated tables.
Database	Saved form config includes config_json, normalized_config_json and feature_map_json.
Assets	Admin assets load only on plugin admin pages.
Assets	Frontend assets load only when a form/filter is present.
Builder	Admin builder supports drag/drop steps, fields, repeaters and grids.
Builder	Builder validates duplicate IDs, broken routes, broken mappings and invalid actions before save.
Fields	All major field types are supported with labels, placeholders, validation and mapping.
Conditions	Fields, HTML, buttons, steps and actions can be conditionally shown/run.
Routing	Steps can route conditionally and block progress when validation/API checks fail.
Validation	All client-side rules are mirrored server-side.
Security	Dynamic nonce refresh works for cached pages.
Security	Honeypot, time-to-submit and rate limiting are available.
Uploads	Uploads happen instantly and are finalized on successful submit.
Actions	Actions on Submit includes update ACF, update user meta, update post meta, taxonomy, product, login, register, email, webhook and redirect.
ACF	Admin can select ACF target object, field group and field, then map source field.
Meta	Admin can select user/post target, meta key and source field.
Products	WooCommerce product mappings use WooCommerce CRUD and only show if WooCommerce is active.
Elementor	Style sections appear based on the selected form feature map.
Tests	Test runner logs expected vs actual results and can clean created test data.
Filters	Sidebar filters support taxonomy, meta, range, pagination and Elementor Loop updates.

Final Implementation Approach

Build the plugin as a reusable platform, not as a single form widget. The WordPress admin drag-and-drop builder should own structure and logic. The database should own the normalized source of truth. Elementor should own styling and display. The frontend engine should run only what the selected form requires. The server should validate everything again before saving data or running submit actions.

This approach allows the plugin to support simple contact forms, complex onboarding flows, mapped profile/product update forms, file upload workflows, login/register flows and sidebar filtering without turning Elementor controls into an unmaintainable nested builder.